

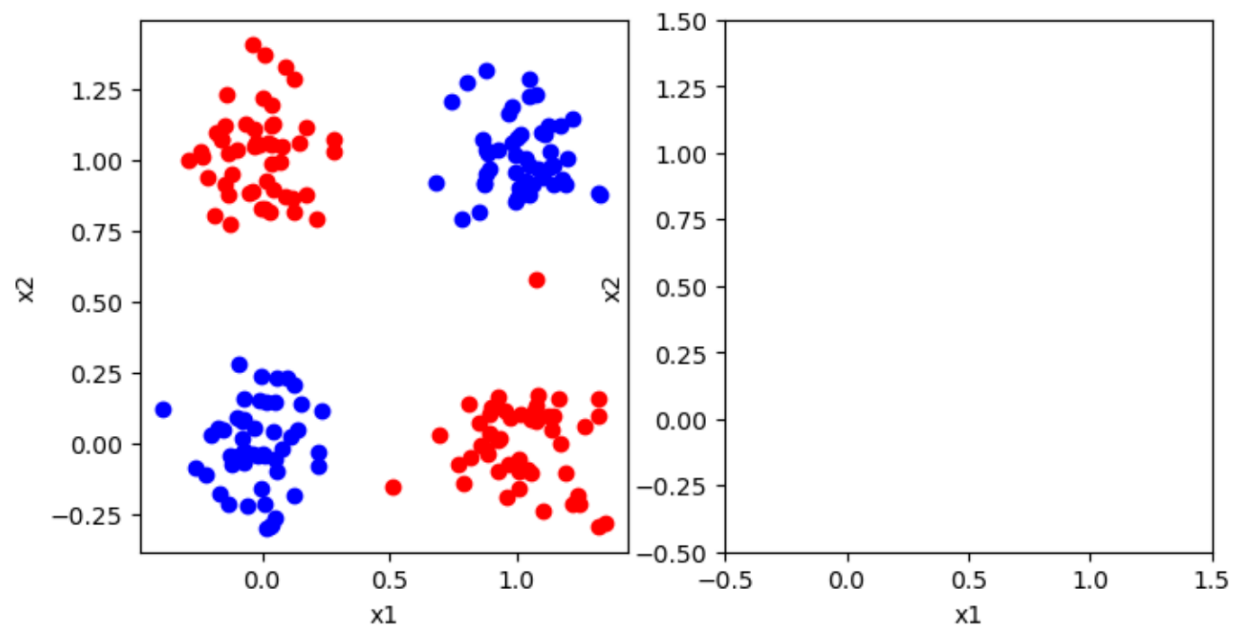
RBF Networks and Prototypical Learning

Introduction

- We have learned how neural networks transform data into new representation to solve non-linear problems.
- We will now look at non-linear classification from the perspective of function approximation.
- Key idea:
 - Transform data in terms of a preselected class of interpolation functions.
- We will also see how RBF networks relate to the field of prototypical learning.

Function approximation

- Recall the XOR example:



General idea

- Let (non-linear) $f_i : \mathbb{R}^d \rightarrow \mathbb{R}, i = 1, \dots, k$
- Look at $\mathbf{x} \in \mathbb{R}^d$. (Usually $k > l$)
- Draw example:

Network

- The function approximation setup described above can be thought of as a network:

Radial basis function (RBF) networks

- General idea:

$$f(\|\mathbf{x} - \mathbf{c}_i\|)$$

- RBF network:

$$f(\mathbf{x}) = \exp\left(-\frac{1}{2\sigma_i^2}\|\mathbf{x} - \mathbf{c}_i\|^2\right)$$

- Activation at node f_i given by distance to $\mathbf{c}_i$.

Estimating the weights

- Any linear method will do!
- Let $\mathbf{y}$ be the desired outputs. Construct matrix $\mathbf{G}$:

$$\mathbf{G} = \begin{bmatrix} \phi(\mathbf{x}_1, \mathbf{c}_1) & \cdots & \phi(\mathbf{x}_1, \mathbf{c}_k) \\ \vdots & & \vdots \\ \phi(\mathbf{x}_N, \mathbf{c}_1) & \cdots & \phi(\mathbf{x}_N, \mathbf{c}_k) \end{bmatrix}_{N \times k}$$

- So:

$$\mathbf{g} = \mathbf{G}\mathbf{w} = \begin{bmatrix} g(\mathbf{x}_1) \\ \vdots \\ g(\mathbf{x}_N) \end{bmatrix}$$

Estimating the weights

- Choose $\mathbf{w}$ such that $J = \|\mathbf{y} - \mathbf{G}\mathbf{w}\|^2$ is minimized.

$$\frac{\partial J}{\partial \mathbf{w}} = 0 \implies \boxed{\mathbf{w} = (\mathbf{G}^\top \mathbf{G})^{-1} \mathbf{G}^\top \mathbf{y}}$$

How to choose the centers?

- **Fixed centres:** Centres selected randomly from the training set.
 - Simple, but not optimum!
- **Training of the centres:** Let $\sigma_i^2, \mathbf{c}_i, w_i, i = 1, \dots, k$, be free parameters learned from the training set.

$$\frac{\partial J}{\partial \mathbf{c}_i} = 0 \implies \mathbf{c}_i^{(\text{new})} = \mathbf{c}_i^{(\text{old})} - \mu \frac{\partial J}{\partial \mathbf{c}_i}$$

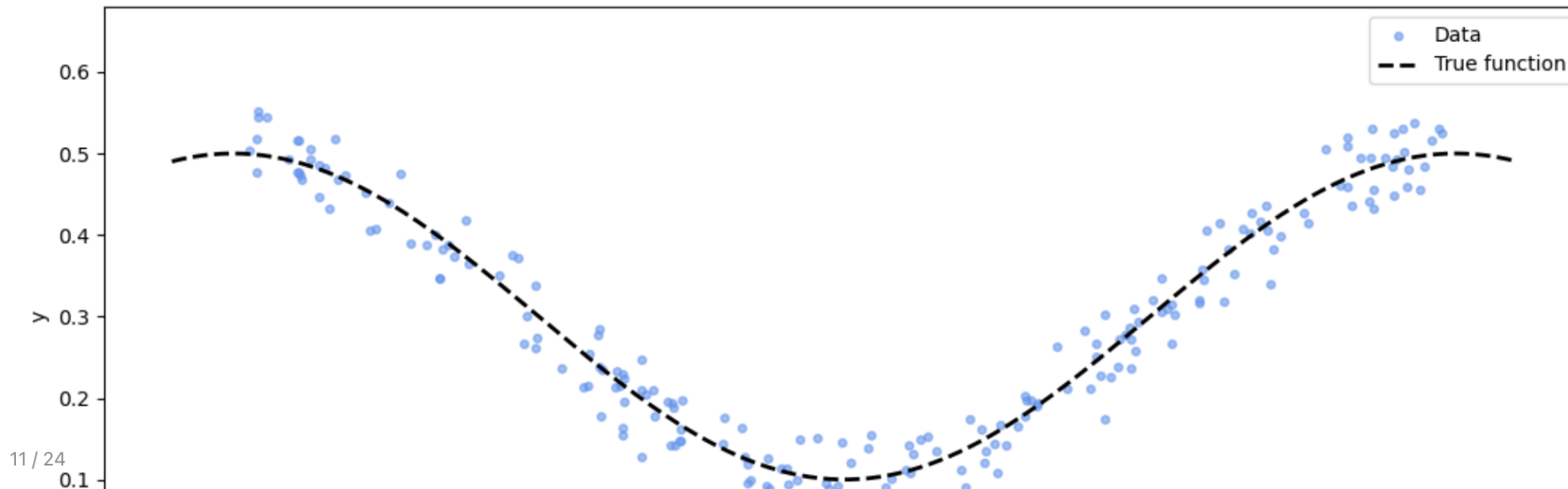
- Choose centres according to how the data are distributed in space.
 - The next two slides show an interactive example – first with centres fixed on a grid, then with centres learned by gradient descent.

Example: function approximation with an RBF network

- Data: $y = f(x) + \varepsilon$, where $f(x) = 0.3 + 0.2 \cos(2\pi x)$ and $x \in [0, 1]$, $N = 200$ samples.
- Model: an RBF network in one dimension,

$$g(x) = \sum_{i=1}^k w_i \exp \left(-\frac{1}{2\sigma^2} (x - c_i)^2 \right),$$

- The weights $\mathbf{w}$ are solved by least squares. First, the centres are fixed on a uniform grid; then they are learned by gradient descent.

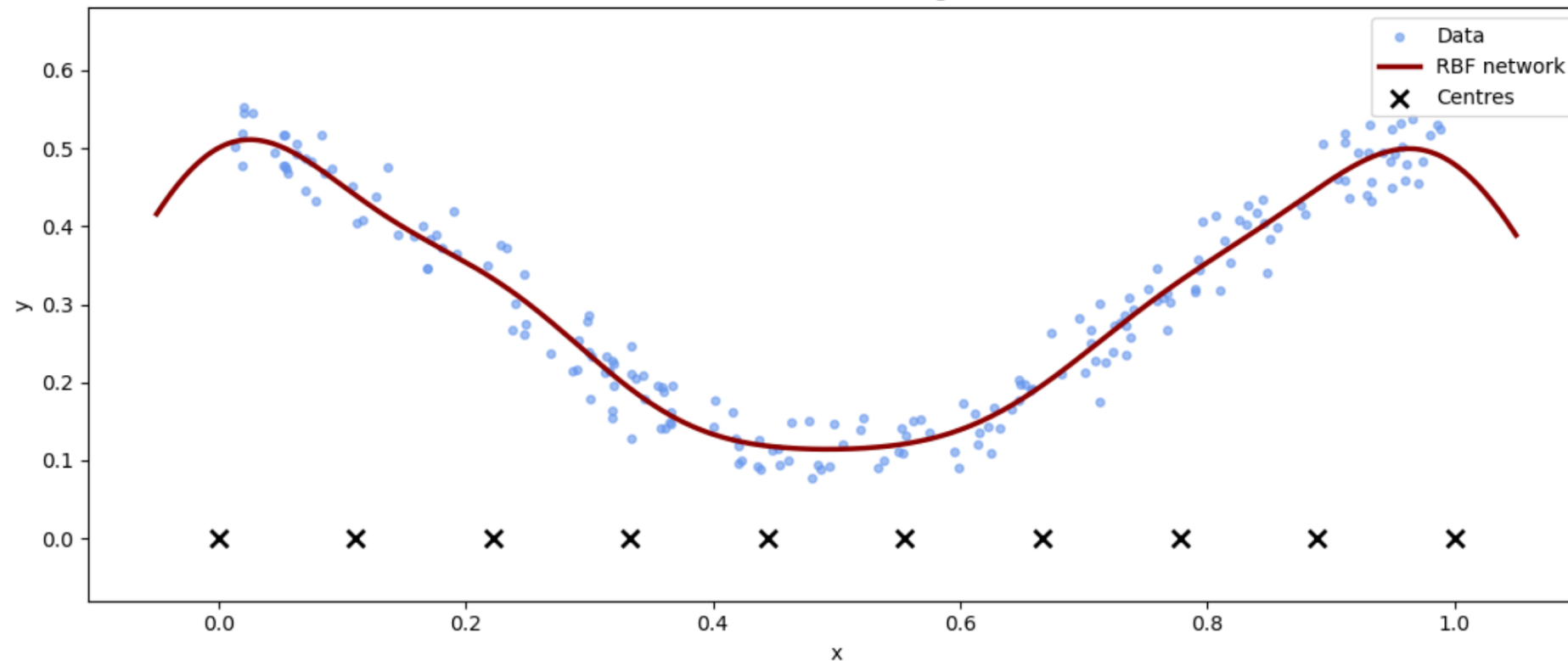


Interactive: fixed centres

Number of centres

Kernel width σ

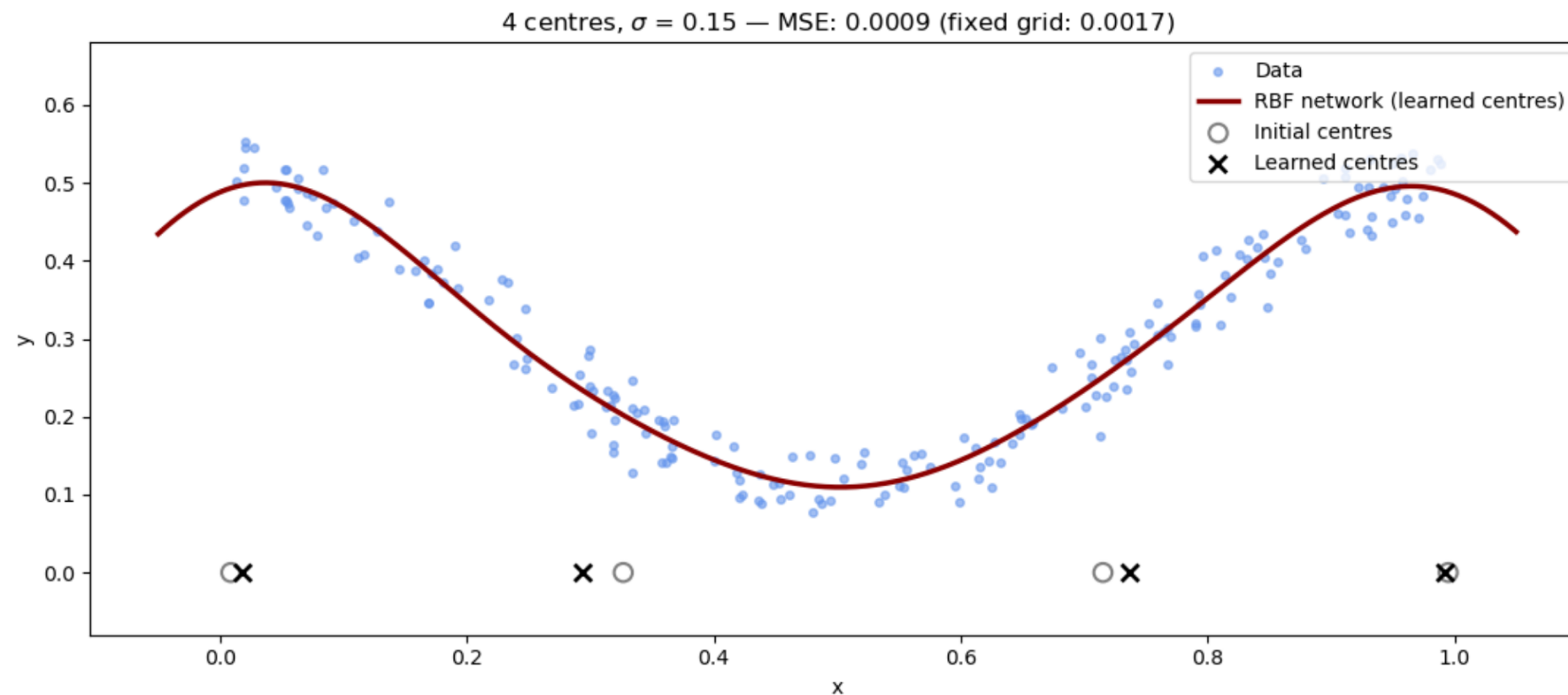
10 centres, $\sigma = 0.10$ — training MSE: 0.0009



Interactive: learned centres

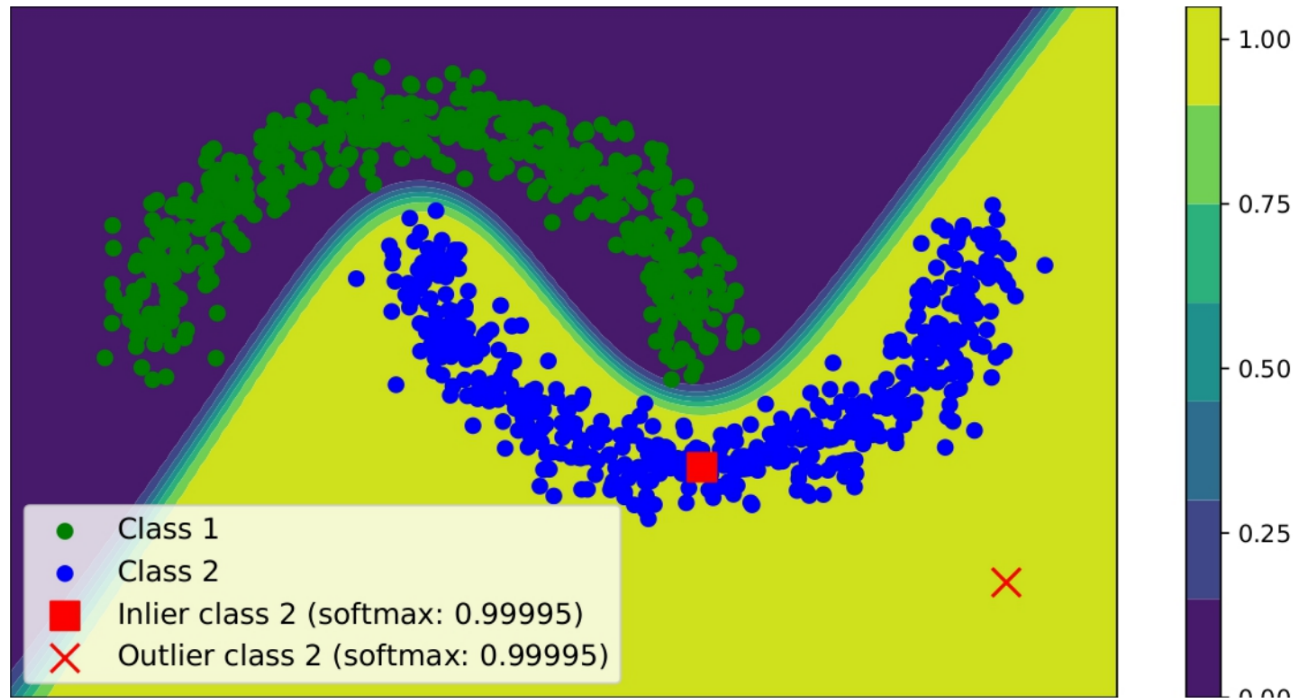
Number of centres

Kernel width σ



Prototypical learning

- RBF networks classify by similarity to reference points – can we make this idea explicit, and use it to build interpretable models?
- Consider the following example:



What is a prototype?

- A **prototype** is a representative example of the data: a point $\mathbf{p}_j \in \mathbb{R}^d$ that is "typical" for (a region of) the input space.
- A prototype-based classifier assigns labels by similarity to a set of prototypes $\mathbf{p}_1, \dots, \mathbf{p}_M$:

$$\hat{y}(\mathbf{x}) = \arg \max_j w_j s(\mathbf{x}, \mathbf{p}_j), \quad s(\mathbf{x}, \mathbf{p}) = \exp \left(-\frac{1}{2\sigma^2} \|\mathbf{x} - \mathbf{p}\|^2 \right)$$

- Prototypes can be defined in the input space or in a latent space!

RBF networks are prototype classifiers

- Comparing with the RBF network,

$$g(\mathbf{x}) = \sum_{i=1}^k w_i \exp \left(-\frac{1}{2\sigma_i^2} \|\mathbf{x} - \mathbf{c}_i\|^2 \right),$$

the centres $\mathbf{c}_i$ play the role of prototypes, the basis functions act as similarities, and the output weights w_i score each prototype.

- Two special cases:
 - Prototypes fixed to (a subset of) the training samples with $\sigma \rightarrow 0$: nearest-prototype classification $\rightarrow$ nearest neighbour classification.
 - Centres learned by gradient descent: prototypes learned from data.

Prototypes in latent space

- Prototypes do not have to live in the input space.
- An encoder maps $\mathbf{x}$ to a representation $\mathbf{z} = \text{enc}(\mathbf{x})$, and the prototypes are defined in this latent space:

$$\hat{y}(\mathbf{x}) = \arg \max_j w_j s(\text{enc}(\mathbf{x}), \mathbf{p}_j)$$

- Similarities in a learned representation space are often more meaningful than similarities between raw samples.

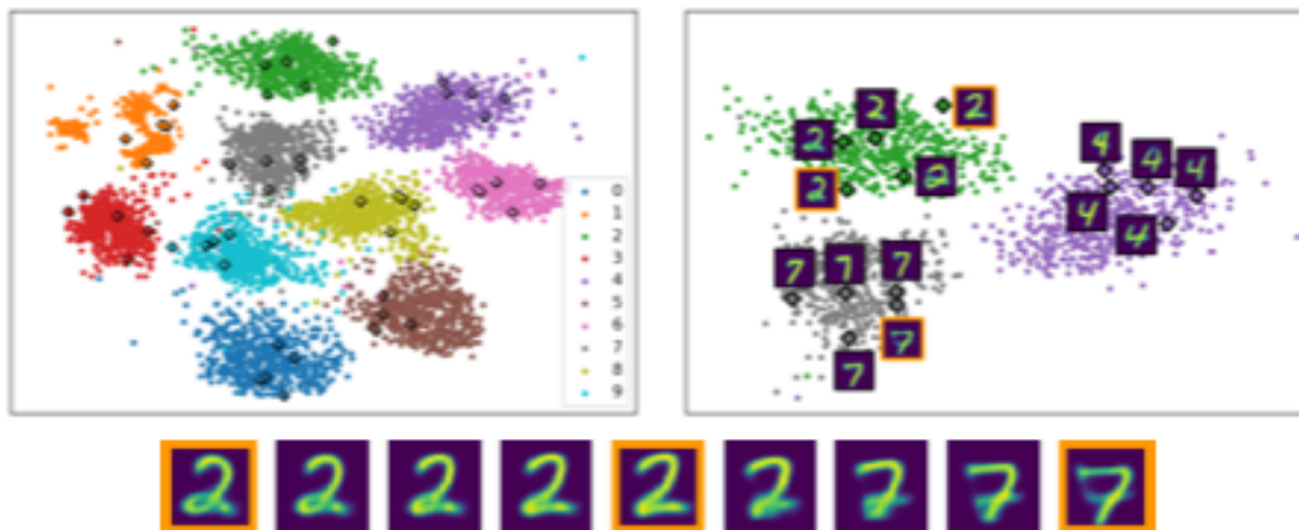


Figure 3: UMAP representations for the prototypical space for MNIST (left), decoded prototypes overlaid for classes 2, 4 and 7 (right), and interpolation between prototypes of the same class (2) and between prototypes of different classes (2-7) (bottom).

Why prototypes? Interpretability

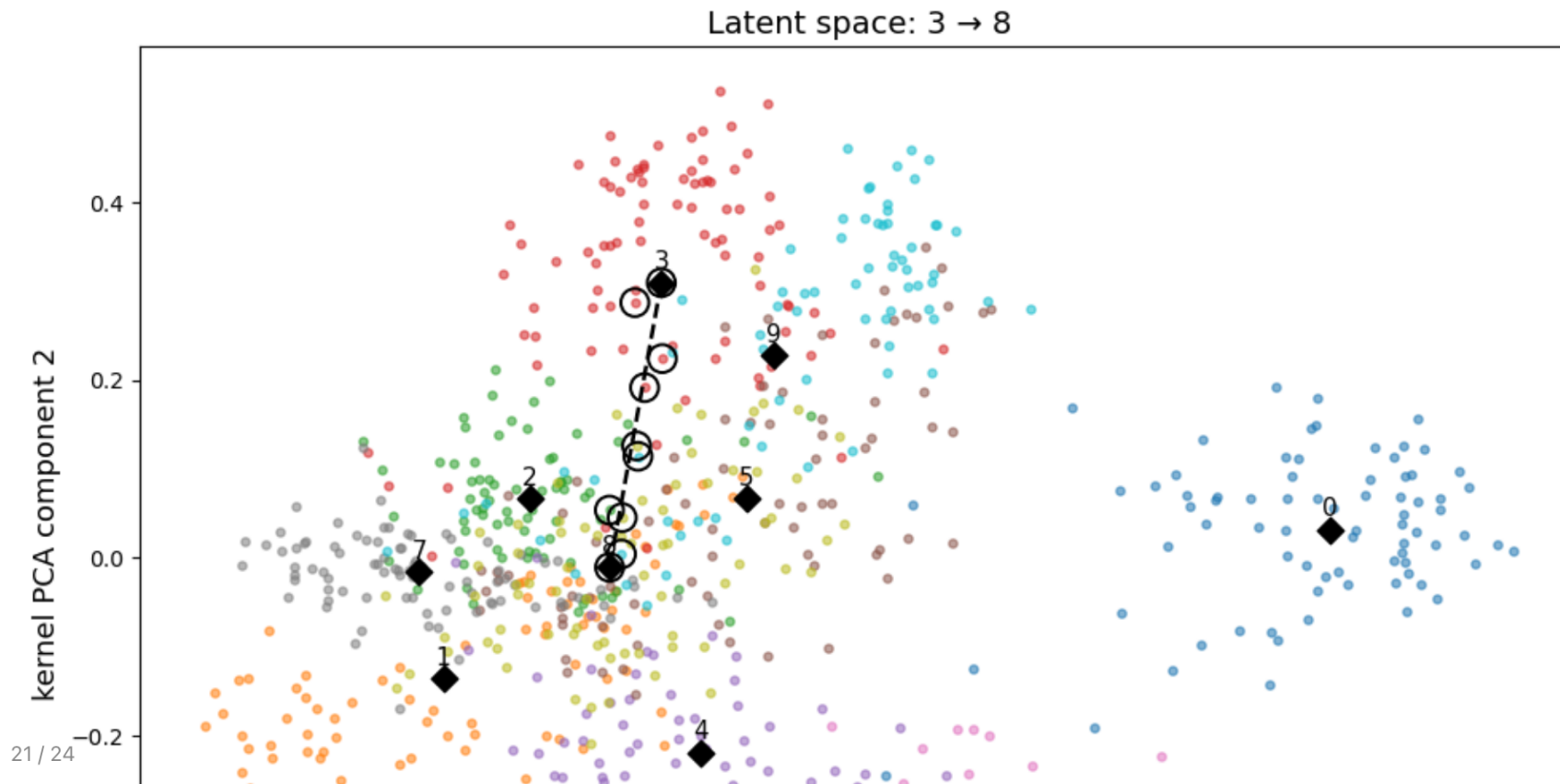
- Classifying by similarity to concrete examples makes the behaviour of the model easier to understand:
 - "This sample is classified as $\hat{y}$ because it resembles prototype $\mathbf{p}_j$."
- Prototypes represent "typical" examples in the data.

Browsing the latent space between prototypes

- Kernel PCA embedding of the digits dataset; the black diamonds are prototypes (the most typical sample of each digit in the embedding).
- Pick two prototypes: the row shows the actual samples encountered when walking from one to the other.

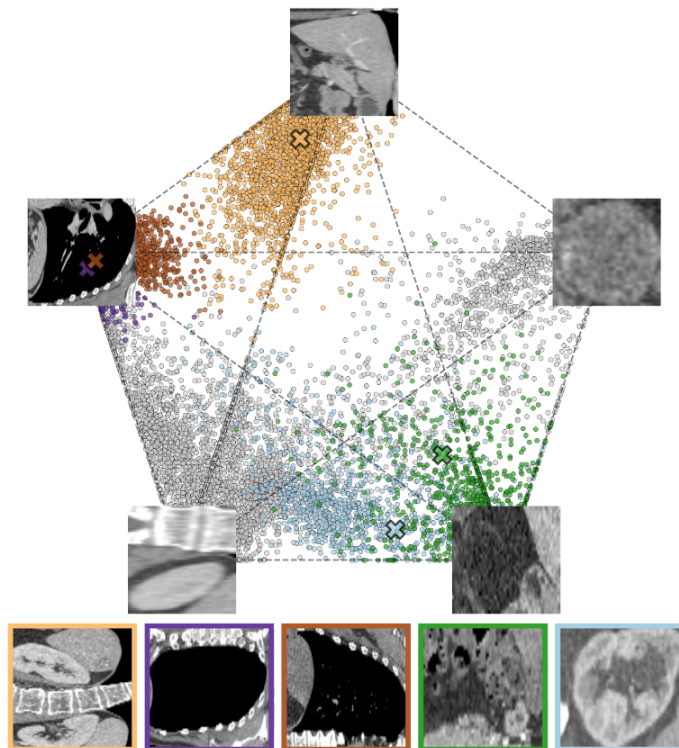
From prototype

To prototype



Prototypes vs. centroids

- A prototype can be an observed sample: a concrete case that can be inspected and compared with new samples.
- A centroid (e.g., a class mean) is a synthesized point that need not correspond to any real sample.
- **Archetypal analysis** (Cutler and Breiman, 1994) summarizes a class by a small set of extreme, "pure" examples – data points on the boundary of the class rather than its centre.



Challenges with prototypical learning

- Curse of dimensionality:
 - In high dimensions, pairwise distances concentrate, so the similarities $s(\mathbf{x}, \mathbf{p}_j)$ become less informative.
 - Covering the input space with prototypes may require exponentially many of them.
- Computational cost:
 - Each prediction requires computing M similarities, each of cost $\mathcal{O}(d)$, and the prototypes must be stored in memory.
- Optimization:
 - Learning $\{\mathbf{c}_i, \sigma_i, w_i\}$ jointly is non-convex, so the result depends on initialization (recall: centres selected randomly from the training set).